

KUBERNETES

PODS

KUBERNETES

Pod es una envoltura que contiene uno o varios contenedores (en la mayoría de los casos un solo contenedor). De forma genérica, un Pod representa un conjunto de contenedores que comparten almacenamiento y una única IP.

- En la mayoría de los casos y siguiendo el principio de un proceso por contenedor, evitamos tener sistemas (como máquinas virtuales) ejecutando docenas de procesos, por lo que lo más habitual será tener un Pod en cuyo interior se define un contenedor que ejecuta un solo proceso.

Algunos ejemplos pueden ser: ejecución en modo **demonio de un servidor web**, ejecución de un servidor de aplicaciones Java, ejecución de una tarea programada, ejecución en modo demonio de un servidor DNS, etc.

- En determinadas circunstancias será necesario ejecutar más de un proceso en el mismo "sistema", como en los casos de procesos fuertemente acoplados, en esos casos, tendremos más de un contenedor dentro del Pod. Cada uno de los contenedores ejecutando un solo proceso, pero pudiendo compartir almacenamiento y una misma dirección IP como si se tratase de un sistema ejecutando múltiples procesos.

Un ejemplo típico de un Pod multicontenedor es un servidor web nginx con un servidor de aplicaciones PHP-FPM, que se implementaría mediante un solo Pod, pero ejecutando un proceso de nginx en un contenedor y otro proceso de php-fpm en otro contenedor.

KUBERNETES

Para crear un Pod podemos hacerlo:

- De forma **imperativa**: mediante `kubectl run pod-nginx --image=nginx`

Un Pod tiene otros muchos parámetros asociados, que en este caso quedarán sin definir o Kubernetes asumirá los valores por defecto. Sin embargo es mucho más habitual trabajar con los objetos de Kubernetes de manera declarativa.

- De forma **declarativa**: definiendo los objetos de forma detallada a través de un fichero en formato YAML.

KUBERNETES

```
apiVersion: v1 # required
kind: Pod # required
metadata: # required
  name: pod-nginx # required
  labels:
    app: nginx
    service: web
spec: # required
  containers:
    - image: nginx:1.16
      name: contenedor-nginx
      imagePullPolicy: Always
```

- **apiVersion:** v1: La versión de la API que vamos a usar.
- **kind:** Pod: La clase de recurso que estamos definiendo.
- **metadata:** Información que nos permite identificar unívocamente el recurso:
 - **name:** Nombre del pod
 - **Labels:** nos permiten etiquetar los recursos de Kubernetes (por ejemplo un pod) con información del tipo clave/valor.
- **spec:** Definimos las características del recurso. En el caso de un Pod indicamos los contenedores que van a formar el Pod (sección containers), en este caso sólo uno.
 - **image:** La imagen desde la que se va a crear el contenedor
 - **name:** Nombre del contenedor.
 - **imagePullPolicy:** Las imágenes se guardan en un registro interno. Se pueden utilizar registros públicos (google o dockerhub son los más usados) y registros privados. La política por defecto es **IfNotPresent**, que se baja la imagen si no está en el registro interno. Si queremos forzar la descarga desde el repositorio externo, tendremos que indicar **imagePullPolicy:Always**.

KUBERNETES

Crear directamente el Pod desde el fichero yaml:	<code>kubectl create -f pod.yaml</code>
Ver el estado en el que se encuentra y si está o no listo:	<code>kubectl get pods</code>
Información sobre los Pods, como por ejemplo, saber en qué nodo del cluster se está ejecutando, ip:	<code>kubectl get pod -o wide</code>
Información más detallada del Pod (equivalente al inspect de docker):	<code>kubectl describe pod pod-nginx</code>
Editar el Pod y ver todos los atributos que definen el objeto, la mayoría de ellos con valores asignados automáticamente por el propio Kubernetes y podremos actualizar ciertos valores:	<code>kubectl edit pod pod-nginx</code>
Eliminar un pod	<code>kubectl delete pod pod-nginx</code>
Obtener directamente los logs al igual que se hace en docker:	<code>kubectl logs pod-nginx</code>
Ejecuta alguna orden adicional en el Pod, podemos utilizar el comando exec, por ejemplo, en el caso particular de que queremos abrir una shell de forma interactiva:	<code>kubectl exec -it pod-nginx -- /bin/bash</code>
Acceder a la aplicación, redirigiendo un puerto de localhost al puerto de la aplicación:	<code>kubectl port-forward pod-nginx 8080:80</code> Y accedemos al servidor web en la url http://localhost:8080 .

KUBERNETES

Imagen de ECR AWS

← → ↻ <https://us-east-1.console.aws.amazon.com/ecr/private-registry/repositories/create?region=us-east-1> ☆

aws [Alt+S] Estados Unidos (Norte de Virginia) voclabs/user3762567=Estudiante_de_prueba @ 6947-3324-1396

Amazon ECR > Registro privado > Repositorios > Crear repositorio privado

Crear repositorio privado

Configuración general

Nombre del repositorio
Proporcione un nombre conciso. En los nombres de repositorios se admiten espacios de nombres, lo que se recomienda para agrupar repositorios similares.

694733241396.dkr.ecr.us-east-1.amazonaws.com/

6 de 256 caracteres máximo (2 mínimo). El nombre debe comenzar con una letra y solo puede contener letras minúsculas, números y caracteres especiales _./.

Mutabilidad de la etiqueta de imagen [Información](#)
Especifique la configuración de mutabilidad de la etiqueta que se va a usar. Cuando se encienda la inmutabilidad de la etiqueta en un repositorio, se impedirá que las etiquetas se sobrescriban.

☒ **Mutable**
Las etiquetas de imagen se pueden sobrescribir.

☐ **Immutable**
Se impide que las etiquetas de imagen se sobrescriban.

Configuración de cifrado

⚠ La configuración de cifrado de un repositorio no se puede cambiar una vez creado el repositorio.

Configuración de cifrado [Información](#)
De forma predeterminada, los repositorios usan el cifrado de Advanced Encryption Standard (AES) estándar del sector. Si lo desea, puede optar por usar una clave almacenada en AWS Key Management Service (KMS) para cifrar las imágenes de tu repositorio.

☒ **AES-256**
Cifrado Advanced Encryption Standard (AES) estándar del sector

☐ **AWS KMS**

KUBERNETES

Imagen de ECR AWS

Comandos de envío para susana

macOS / Linux | Windows

Asegúrese de tener instalada la versión más reciente de AWS CLI y Docker. Para obtener más información, consulte [Empezar a utilizar Amazon ECR](#).

Siga los siguientes pasos a fin de autenticar y enviar una imagen a su repositorio. Para obtener métodos de autenticación de registro adicionales, incluido el ayudante de credenciales de Amazon ECR, consulte [Autenticación del registro](#).

1. Recupere un token de autenticación y autentique su cliente de Docker en el registro. Use AWS CLI:

```
aws ecr get-login-password --region us-east-1 | docker login --username AWS --password-stdin 694733241396.dkr.ecr.us-east-1.amazonaws.com
```

Nota: Si recibe un error al utilizar AWS CLI, asegúrese de tener instaladas las últimas versiones de AWS CLI y Docker.
2. Cree una imagen de Docker con el siguiente comando. Para obtener información sobre cómo crear un archivo de Docker desde cero, consulte las instrucciones [aquí](#). Puede omitir este paso si ya se creó la imagen:

```
docker build -t susana .
```

Imagen
3. Cuando se complete la creación, etiquete la imagen para poder enviarla a este repositorio:

```
docker tag susana:latest 694733241396.dkr.ecr.us-east-1.amazonaws.com:susana:latest
```

Repositorio
4. Ejecute el siguiente comando para enviar esta imagen al repositorio de AWS recién creado:

```
docker push 694733241396.dkr.ecr.us-east-1.amazonaws.com:susana:latest
```

Cerrar

de la imagen. Para acceder, haga clic

Analizar Ver comandos de envío

< 1 >

URI de imagen Resumir

- En el repositorio he usado susana:myldap pq si no le ponía a la imagen de nombre latest, así que he usado susana:myldap en la zona de repositorios

KUBERNETES

Imagen de ECR AWS

Previamente creo las credenciales de aws con aws configure.

Necesito crear un secreto:

```
susana@minikube2025:~$ kubectl create secret docker-registry ecr-secret --docker-server=694733241396.dkr.ecr.us-east-1.amazonaws.com --docker-username=AWS --docker-password=$(aws ecr get-login-password --region us-east-1)
```

Y posteriormente asociarlo con mi cuenta:

```
susana@minikube2025:~$ kubectl patch serviceaccount default \
-p '{"imagePullSecrets": [{"name": "ecr-secret"}]}'
serviceaccount/default patched
```

En Kubernetes, **cada pod utiliza un ServiceAccount** para interactuar con el sistema, y el **ServiceAccount** por defecto es el llamado **default**. Cuando creas un pod, a menos que especifiques un **ServiceAccount** diferente, Kubernetes automáticamente le asigna este.

Si no configuras un secreto de autenticación para el **ServiceAccount**, el pod no tendrá permisos para acceder a registros de contenedores privados (como ECR) y, por lo tanto, no podrá descargar las imágenes necesarias, lo que causa errores como **ImagePullBackOff**.